Weather App

Roteiro de Estudo da Arquitetura

Material para impressão e estudo aprofundado
Java 21 · Spring Boot 4 · Hexagonal · Docker · OpenWeather

Objetivo: compreender a aplicação de ponta a ponta —
ler, interpretar, depurar e evoluir com autonomia.

Inclui: confirmação do fluxo, mapas, diagramas, códigos reais,
módulos 1–12, proposta de branch de estudos e Módulo 1 completo.

Baseado no estado real do repositório.
Diferencie sempre: (1) fato no código (2) inferência (3) melhoria ainda não existente.
Regenerar: cd docs && python3 generate_study_roadmap.py

0. Como usar este roteiro

Este PDF organiza o estudo do Weather App em módulos progressivos. Não é um manual para implementar features novas: o foco é compreender o que já existe — por que cada peça existe, como se conecta e quais alternativas existem.

Método sugerido

- Leia um módulo por sessão (comece pelo Módulo 1, completo neste PDF).
- Abra os arquivos reais citados enquanto lê.
- Responda às perguntas de revisão antes de avançar.
- Faça os exercícios de leitura sem alterar o código (ou use branch de estudos).

Perguntas-guia para cada conceito

- O que é? — definição e terminologia
- Onde está? — arquivo, classe, pacote, camada
- O que faz? — responsabilidade prática
- Por que existe? — problema que resolve
- Como funciona? — sintaxe e interações
- Como se conecta? — relação com o fluxo completo
- Quais alternativas? — trade-offs
- Como se relaciona com Node.js? — analogia cuidadosa (sem forçar)

1. Confirmação do fluxo que você já estudou

Você descreveu o fluxo de uma requisição que consulta a OpenWeather, trata JSON/erros e devolve uma resposta processada. Isso está correto para o caso GET com cache miss. Ressalvas importantes:

- Nem toda requisição chama a OpenWeather. O WeatherService é cache-aside: primeiro PostgreSQL; só se a cidade não existir no banco a API externa entra.
- Há três fluxos HTTP: GET /weather (lista o cache no Postgres, sem OpenWeather); GET /weather/{city} (busca + possível OpenWeather); POST /weather (cria no banco; NÃO passa pela OpenWeather).
- Erros têm famílias distintas: input inválido → 400 (@Valid); cidade inexistente → 404 (CityNotFoundException); falhas inesperadas (ex.: key inválida, DB) → hoje 500 genérico (ainda sem tratamento específico de erro externo).

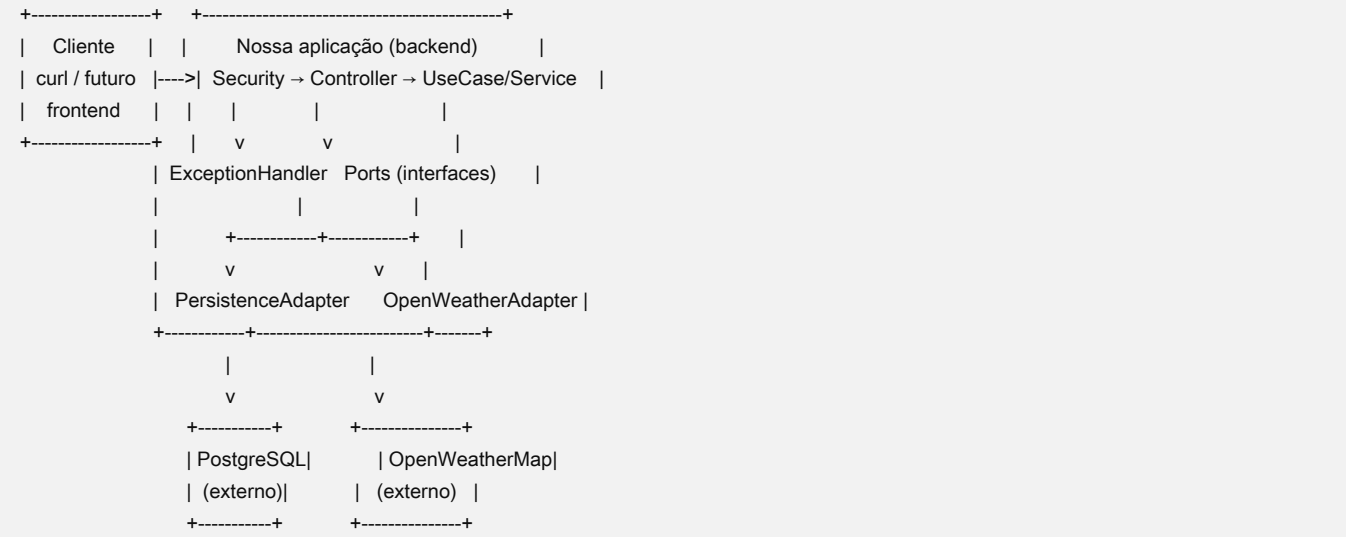
FATO no código: a decisão 'DB primeiro, API depois' está em WeatherService.execute(String city), não no controller.

2. Mapa inicial da aplicação

Problema que a aplicação resolve

API HTTP de clima para estudo: dado o nome de uma cidade, devolver temperatura e descrição, preferindo cache local e buscando na internet só quando necessário. Não é um produto completo (sem frontend, sem auth real, testes quase só smoke).

Diagrama — mapa de componentes



Camadas (intenção hexagonal)

- core.domain + core.ports — o 'quê' (Weather, contratos)
- application.services — orquestração dos casos de uso (WeatherService)
- infrastructure.* — HTTP, JPA, OpenWeather, Security, erros, validação

Tecnologias e papéis

- Java 21 — linguagem, records, tipagem estática
- Spring Boot 4 — HTTP, DI, Security, Validation, JPA
- PostgreSQL 15 + Flyway — persistência e migrations
- OpenWeatherMap — fonte externa de clima
- Docker Compose — sobe app + banco juntos
- Maven Wrapper — dependências e build

Nosso vs externo

- Nosso: controllers, service, ports, adapters, security, validation, ApiError
- Externo: OpenWeatherMap e o processo/container PostgreSQL

Analogia Node.js (cuidado — não são iguais)

- @RestController ≈ rotas Express/Fastify (mas com DI do Spring)
- WeatherService + ports ≈ service + interface (estilo NestJS)
- Adapter JPA ≈ Prisma/TypeORM atrás de uma interface
- RestClient ≈ axios/fetch tipado
- @ControllerAdvice ≈ middleware central de erro

Diferença: em Node a DI costuma ser manual ou NestJS; no Spring o container cria e injeta os beans sozinho. Não assuma que 'parecido' = 'igual'.

3. Caminho de uma requisição real

Exemplo concreto: GET /weather/Porto (cache miss)

- 1. Cliente: curl http://localhost:8080/weather/Porto
- 2. Tomcat + Security: rota pública (permitAll)
- 3. WeatherController: extrai city do path (@PathVariable)
- 4. GetWeatherUseCase → WeatherService: orquestra a regra
- 5. WeatherRepositoryPort → PersistenceAdapter → JPA: SELECT vazio
- 6. ExternalWeatherPort → OpenWeatherExternalAdapter: HTTP na OpenWeather
- 7. JSON → OpenWeatherResponse → Weather (infrastructure → domínio)
- 8. Repository save: INSERT em tb_weather
- 9. Controller: 200 + JSON do Weather

Segunda chamada ao mesmo path: passos 6–7 NÃO ocorrem (cache hit). POST inválido para no @Valid → GlobalExceptionHandler → 400 (service nem roda).

Diagrama de sequência (cache miss vs hit)

```
CACHE MISS (1ª vez Porto)            CACHE HIT (2ª vez Porto)
---------------------------          ---------------------------
Cliente → Controller                 Cliente → Controller
    → Service                            → Service
    → DB findByCity (vazio)                  → DB findByCity (achou)
    → OpenWeather fetch                      → return Weather
    → DB save                            → 200 JSON
    → return Weather
    → 200 JSON
```

Código real — decisão no WeatherService

Arquivo: backend/.../application/services/WeatherService.java

```
@Override
public Weather execute(String city) {
    return weatherRepositoryPort.findByCity(city)
        .orElseGet(() -> {
            Weather externalWeather = externalWeatherPort.fetchWeather(city)
                .orElseThrow(() -> new CityNotFoundException(city));
            return weatherRepositoryPort.save(externalWeather);
        });
}
```

Código real — entrada HTTP (controller fino)

Arquivo: backend/.../infrastructure/controllers/WeatherController.java

```
@GetMapping("/{city}")
public ResponseEntity<Weather> getWeather(@PathVariable String city) {
    Weather weather = getWeatherUseCase.execute(city);
    return ResponseEntity.ok(weather);
}

@PostMapping
public ResponseEntity<Weather> createWeather(
        @Valid @RequestBody WeatherRequest request) { ... }
```

Porta de saída — contrato (sem saber OpenWeather)

Arquivo: backend/.../core/ports/ExternalWeatherPort.java

```
public interface ExternalWeatherPort {
    Optional<Weather> fetchWeather(String city);
}
```

4. Estrutura dos módulos (ordem recomendada)

Estude nesta ordem. Cada módulo deve responder: o quê / onde / por quê / como / Node. Ritmo sugerido: 1 módulo por sessão.

- Módulo 1 — Visão geral da aplicação. Por quê: Mapa mental antes dos detalhes — COMPLETO neste PDF.
- Módulo 2 — Linguagem, sintaxe e tipagem (Java). Por quê: Ler o código sem 'mágica'.
- Módulo 3 — Framework, HTTP e ciclo de vida. Por quê: Seguir uma request no Spring.
- Módulo 4 — Integração OpenWeather. Por quê: Caso de estudo da API externa.
- Módulo 5 — Arquitetura hexagonal (crítica). Por quê: O que está bem e o que é simplificação.
- Módulo 6 — Clean Code, SOLID e design. Por quê: Julgar decisões no código real.
- Módulo 7 — Banco de dados e persistência. Por quê: Entity vs Domain, Flyway, JPA.
- Módulo 8 — Pacotes e estrutura do projeto. Por quê: Localizar responsabilidades.
- Módulo 9 — Docker, Maven e ferramentas. Por quê: Como o sistema sobe e se configura.
- Módulo 10 — Testes — visão introdutória. Por quê: Como travaríamos o conhecimento (sem implementar ainda).
- Módulo 11 — Possível frontend. Por quê: Consumo HTTP superficial.
- Módulo 12 — Revisão e consolidação. Por quê: Glossário, mapas, autoavaliação.

Checklist mental por módulo

- Ler a visão do módulo
- Abrir 1–3 arquivos reais citados
- Responder: o que / onde / por quê / como se conecta
- Fazer analogia Node.js (só se houver equivalência real)
- Responder perguntas de revisão
- Só então avançar

5. Proposta: branch de estudos

Sugestão (criar só após sua aprovação): manter main limpa para desenvolvimento e uma branch study/guide para material didático.

```
main         → código de desenvolvimento
study/guide  → mesma base + docs/comentários de estudo

git checkout main
git checkout study/guide
git checkout study/guide && git merge main   # atualizar estudos
```

Onde colocar o material

- Principal: docs/study/01-visao-geral.md … 12-consolidacao.md
- Opcional: docs/study/REQUEST-WALKTHROUGH.md

Comentários no código — riscos

- Prós de comentar nos .java: contexto imediato ao ler a classe
- Contras: merge ruidoso com main; risco de commit acidental na main
- Recomendação: docs em docs/study/; no código só TO LEARN curtos (já existem alguns), sem ensaios longos nas classes

Nada desta proposta foi aplicada automaticamente. Branch e arquivos só depois da sua aprovação explícita.

6. Agenda detalhada — Módulos 2 a 12

O Módulo 1 está completo na seção seguinte. Os módulos 2–12 são o roteiro do que estudar em sessões futuras (ainda a aprofundar).

Módulo 2 — Java: linguagem e tipagem

- Classes, interfaces, records, construtores (Weather, WeatherEntity)
- Tipagem estática vs JS; Optional; genéricos
- Modificadores public/private/protected; final
- Annotations (@Service, @Component…) vs decorators Nest/TS
- Exceções (CityNotFoundException) vs throw em Node
- DI por construtor — ciclo de vida dos beans Spring
- Separar: o que é Java puro vs o que é Spring

Módulo 3 — Framework, HTTP e ciclo de vida

- WeatherController: @RestController, @GetMapping, @PostMapping
- @PathVariable, @RequestBody, ResponseEntity, status 200/201
- WeatherRequest + @Valid + Bean Validation → 400
- Serialização JSON (Jackson) do record Weather
- GlobalExceptionHandler / ApiError
- Exercício: acompanhar GET e POST passo a passo

Módulo 4 — OpenWeather

- Por que a responsabilidade está no adapter, não no service
- RestClient + OpenWeatherConfig + WEATHER_API_KEY
- OpenWeatherResponse (DTO) vs Weather (domínio)
- O que o código faz hoje em 404 vs 401/timeout (fato vs melhoria)
- Profile mock vs !mock

Código real — esqueleto do adapter

Arquivo: .../adapters/out/external/OpenWeatherExternalAdapter.java

```
@Component
@Profile("!mock")
public class OpenWeatherExternalAdapter implements ExternalWeatherPort {
    // RestClient + apiKey injetados
    public Optional<Weather> fetchWeather(String city) {
        // GET /weather?q=...&appid=...&units=metric&lang=pt_br
        // 404 → Optional.empty()
        // sucesso → Optional.of(new Weather(city, temp, desc, now))
    }
}
```

Módulo 5 — Hexagonal (análise crítica)

- Ports de entrada (use cases) e saída (repository, external)
- Direção das dependências: infrastructure → core, não o contrário
- O que está bem vs simplificações (controller devolve domain)

```
Dependências desejadas:
  Controller ──usa──▸ UseCase (port in)
  Service    ──usa──▸ RepositoryPort / ExternalPort
  Adapters   ──implementam──▸ Ports
  Domain     ──não conhece──▸ Spring / JPA / OpenWeather
```

Módulo 6 — Clean Code e SOLID

- SRP: controller fino vs service orquestrador
- ISP: GetWeatherUseCase vs SaveWeatherUseCase no controller
- DIP: service depende de ports, não de JPA/OpenWeather
- Custos de abstração desnecessária — quando NÃO criar interface

Módulo 7 — Persistência

- PostgreSQL neste projeto; env vars no Compose
- WeatherEntity vs Weather vs WeatherRequest
- Flyway V1__create_tb_weather.sql; ddl-auto=none
- UNIQUE city; 409 ainda NÃO implementado

Módulo 8 — Árvore de pacotes

```
weather-app/
  docker-compose.yml   .env.example   README.md
  docs/                (PDFs + geradores Python)
  backend/
    pom.xml  dockerfile  scripts/start.sh
    src/main/java/com/weather_app/
      core/  application/  infrastructure/
    src/main/resources/
      application.properties
      db/migration/V1__create_tb_weather.sql
```

Módulo 9 — Docker e ferramentas

- Imagem vs container; portas; rede Compose
- .env / WEATHER_API_KEY; rebuild vs up
- Logs: docker logs weather-backend
- Maven: ./mvnw compile / test / package

Módulo 10 — Testes (visão, sem implementar agora)

- Unitário do WeatherService com mocks dos ports
- @WebMvcTest para 400/404/201
- Por que hexagonal facilita testes
- Comparar com Jest + supertest / Nest testing

Módulo 11 — Frontend possível

- fetch/axios → mesmos endpoints e status
- Loading / erro / ApiError.fieldErrors
- CORS e auth como decisões futuras (não existem no app hoje)

Módulo 12 — Consolidação (entregáveis)

- Mapa da arquitetura com suas palavras
- Diagrama do fluxo completo
- Glossário + tabela Java ↔ Node
- Comandos úteis; pontos fortes/limitações/melhorias
- Roteiro de autoavaliação

7. Módulo 1 completo — Visão geral da aplicação

O que é este projeto?

Uma API HTTP de clima para estudo. Resolve: dado o nome de uma cidade, devolver temperatura/descrição, preferindo cache local e buscando na internet só quando necessário.

Como está organizado?

```
backend/src/main/java/com/weather_app/
  core/domain          Weather, CityNotFoundException
  core/ports           Get/Save use cases + Repository + External ports
  application/services WeatherService
  infrastructure/
    controllers        REST, WeatherRequest, ApiError, GlobalExceptionHandler
    config             SecurityConfig, OpenWeatherConfig
    adapters/out/
      persistence      WeatherEntity, JpaRepository, PersistenceAdapter
      external         OpenWeather* + WeatherMockExternalAdapter
```

Componentes no fluxo GET

- WeatherController — entrada HTTP
- GetWeatherUseCase / GetAllWeathersUseCase / WeatherService
- WeatherRepositoryPort / PersistenceAdapter — Postgres
- ExternalWeatherPort / OpenWeatherExternalAdapter — API (só get-by-city)
- GlobalExceptionHandler — tradução de falhas em JSON HTTP

Jornada da requisição (visão única)

```
GET /weather (findAll — só cache local):
  Controller → GetAllWeathersUseCase → Service.execute()
          → RepositoryPort.findAll() → Adapter (Entity→Weather) → 200 JSON []

GET /weather/{city}:
  Cliente → Security → Controller → WeatherService
      ├─ findByCity (Postgres)
      │    hit  → retorna Weather
      │    miss → fetchWeather (OpenWeather)
      │             empty → CityNotFoundException → 404 ApiError
      │             ok    → save (Postgres) → retorna Weather
  → 200 JSON

POST /weather + body inválido:
  → @Valid falha → MethodArgumentNotValidException → 400 + fieldErrors
  (WeatherService NÃO executa)
```

Hexagonal: fato vs simplificação

- Presente: ports, adapters, domínio sem JPA/OpenWeather
- Simplificações: controller devolve record de domínio; um service implementa dois use cases; testes quase ausentes

Comparação rápida Node.js

Aqui	Node (aproximação)
@RestController	rotas Express/Fastify
WeatherService + ports	service + interface/DI
WeatherRepositoryPort	repository interface
Adapter JPA	Prisma/TypeORM impl

```
Adapter OpenWeather          |  axios client isolado
@ControllerAdvice / ApiError |  error middleware
record Weather               |  type + objeto imutável
```

Resumo do Módulo 1

- App = API de clima com cache Postgres + fallback OpenWeather
- GET e POST são fluxos diferentes
- Externos: DB e OpenWeather; o resto é nosso
- Hexagonal útil, não 'pura de livro'

Conceitos essenciais

Cache-aside · Port/Adapter · Domínio vs Infrastructure · Request lifecycle · Boundary com sistema externo

Perguntas de revisão — Módulo 1

- 1. Por que a segunda chamada a /weather/Porto não precisa da OpenWeather?
- 2. Qual componente decide 'DB primeiro, API depois'?
- 3. O POST usa a OpenWeather? Por quê?
- 4. Cite uma coisa nossa e uma externa no fluxo GET.
- 5. Em Node, o que mais se parece com ExternalWeatherPort + OpenWeatherExternalAdapter?

Exercício de leitura (sem mudar código)

Abra WeatherService.execute(String city) e, em voz alta ou no papel, numere os 3 passos do comentário Smart Flow. Digue qual classe concreta o Spring injeta hoje em cada port — com profile padrão e com profile mock.

Espaço para suas respostas (escreva à mão)

1.

2.

3.

4.

5.

8. Comandos úteis (referência rápida)

Docker

```
docker compose build backend
docker compose up -d
docker compose logs -f backend
docker compose down
```

API — smoke e cenários de estudo

```
curl -s http://localhost:8080/hello

# list all cached (DB only)
curl -s http://localhost:8080/weather | python3 -m json.tool

# cache miss / hit
curl -s http://localhost:8080/weather/Porto | python3 -m json.tool

# 404 ApiError
curl -s http://localhost:8080/weather/CidadeXYZ | python3 -m json.tool

# 400 validation
curl -s -X POST http://localhost:8080/weather \
  -H 'Content-Type: application/json' \
  -d '{"city":"","temperature":null,"description":""}' | python3 -m json.tool

# 201 create
curl -s -i -X POST http://localhost:8080/weather \
  -H 'Content-Type: application/json' \
  -d '{"city":"Coimbra","temperature":22.0,"description":"Sol"}'
```

Build local

```
cd backend && ./mvnw compile
cd backend && ./mvnw test
```

Regenerar este PDF

```
cd docs
python3 generate_study_roadmap.py
# ou: .venv/bin/python generate_study_roadmap.py
```

9. Estado atual — o que estudar vs o que NÃO inventar

Já implementado (pode estudar no código)

- Hexagonal: GET /weather (findAll), GET /weather/{city}, POST /weather; Flyway; OpenWeather; mock profile
    - GetAllWeathersUseCase + WeatherRepositoryPort.findAll() — adapter traduz List<WeatherEntity> → List<Weather> (stream/map/toList)
- ApiError: 404 cidade, 400 validação (+ fieldErrors), 500 fallback
- Docker Compose + WEATHER_API_KEY via .env
- Documentação README + backend/documentation.md alinhadas

Ainda NÃO implementado

- 409 cidade duplicada; 502 falha OpenWeather; 400 JSON malformado dedicado
- Testes automatizados além do smoke; TTL de cache; Actuator; OpenAPI; JWT
- Frontend; Kafka; Cucumber; CI

Ao estudar, diferencie sempre: (1) fato no código, (2) inferência, (3) melhoria recomendada mas não existente.

10. Resultado esperado e próximos passos

Ao concluir o roteiro completo, você deve conseguir, com suas próprias palavras: explicar a arquitetura; acompanhar uma requisição de ponta a ponta; justificar as principais decisões técnicas; e dizer como cada tecnologia participa do sistema — incluindo limitações conscientes do MVP de estudo.

Próximo passo sugerido agora

- Imprimir este PDF e responder as perguntas do Módulo 1 à mão
- Decidir se quer a branch study/guide + docs/study/
- Avançar para o Módulo 2 (Java/sintaxe com exemplos reais do projeto)

Fim do roteiro de partida — bom estudo.